StoreIntel: AI-Powered Real-Time Store Intelligence & Customer Analytics

Purplle Tech Challenge 2026 — Round 2 Technical Proposal

Submitted by **Shivam Giri** · Full-Stack & AI Engineer (Individual) · June 2, 2026

GITHUB REPOSITORY

github.com/shivamgiri068/StoreIntel

Executive Summary

Physical retail operates with a critical data blindspot. E-commerce platforms track click-through rates, session dwell times, and conversion funnels in real time — yet brick-and-mortar managers still rely on manual log sheets and static POS records. **StoreIntel bridges this gap** by converting existing CCTV security camera feeds into spatial customer journey metrics, dwell-time heatmaps, and operational anomaly alerts — with zero additional hardware.

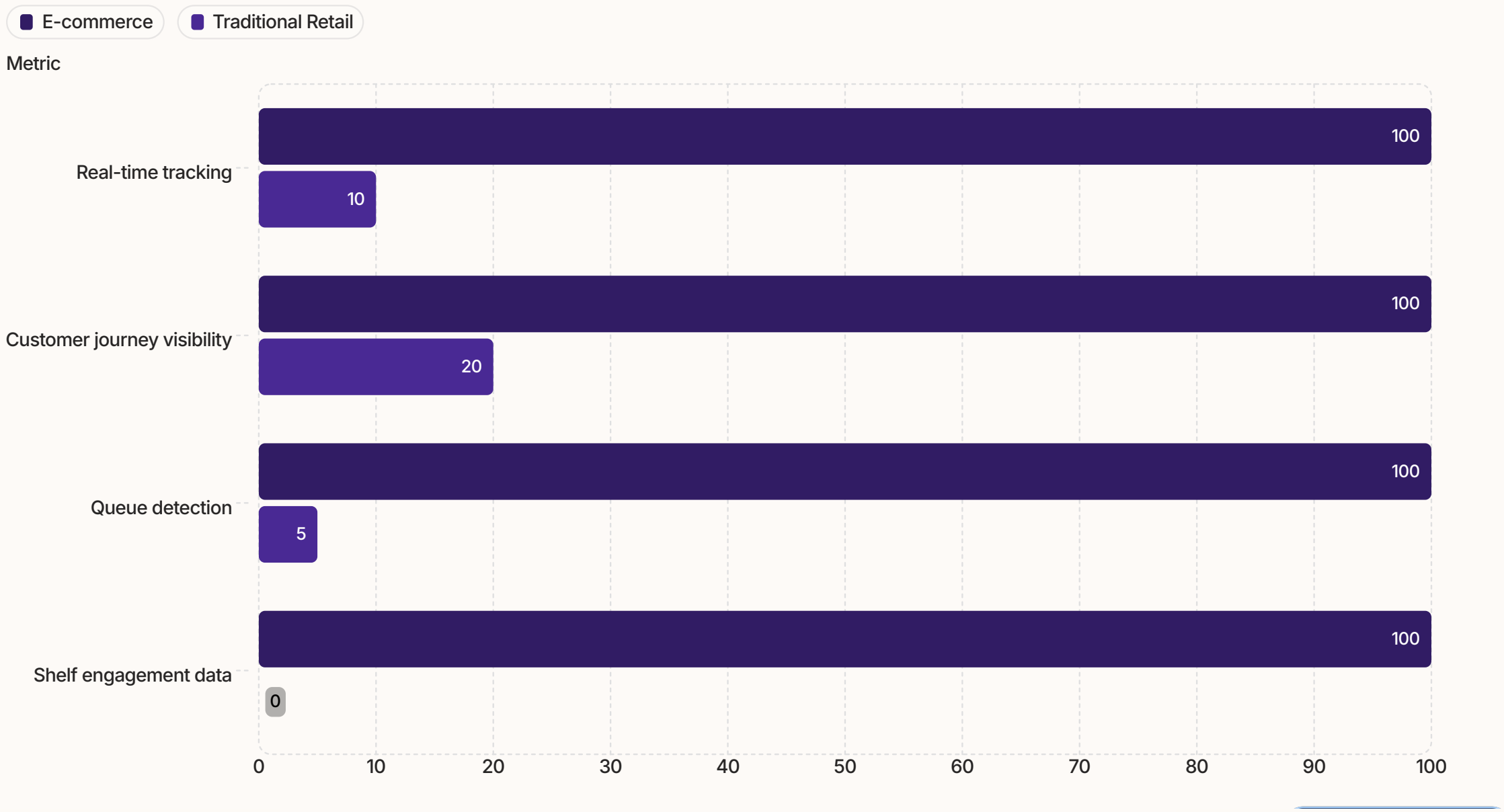
The Problem

- No visibility into in-store customer paths
- Manual headcounts are slow and error-prone
- Queue congestion goes undetected until complaints arise
- Shelf engagement data is entirely absent

The StoreIntel Solution

- Leverages existing CCTV infrastructure — no new hardware
- Real-time person tracking at 30+ FPS on edge CPUs
- Zone-based spatial analytics with polygon containment
- Automated anomaly detection for queues and loitering

Data Visibility Comparison



Technology Stack

StoreIntel is engineered for edge deployment — lightweight, self-contained, and production-ready via a single Docker Compose command. Each layer is chosen for performance, minimal overhead, and seamless integration.

AI & Tracking

YOLOv8 Nano + ByteTrack with Kalman filter association. Runs at 30+ FPS on edge CPUs with resilient track ID persistence.

API Framework

Python FastAPI — high-throughput async request serving with Pydantic validation for strict payload integrity.

Local Database

SQLite — zero-configuration, self-contained SQL engine. No external database orchestration required.

Visualization

Streamlit Dashboard — rapid operational UI with native Plotly heatmaps, funnel charts, and zone metrics.

Orchestration

Docker Compose — multi-container isolation with reproducible environments. Deploy the full stack via one command.

Edge Detection & Multi-Object Tracking

YOLOv8 Person Filter

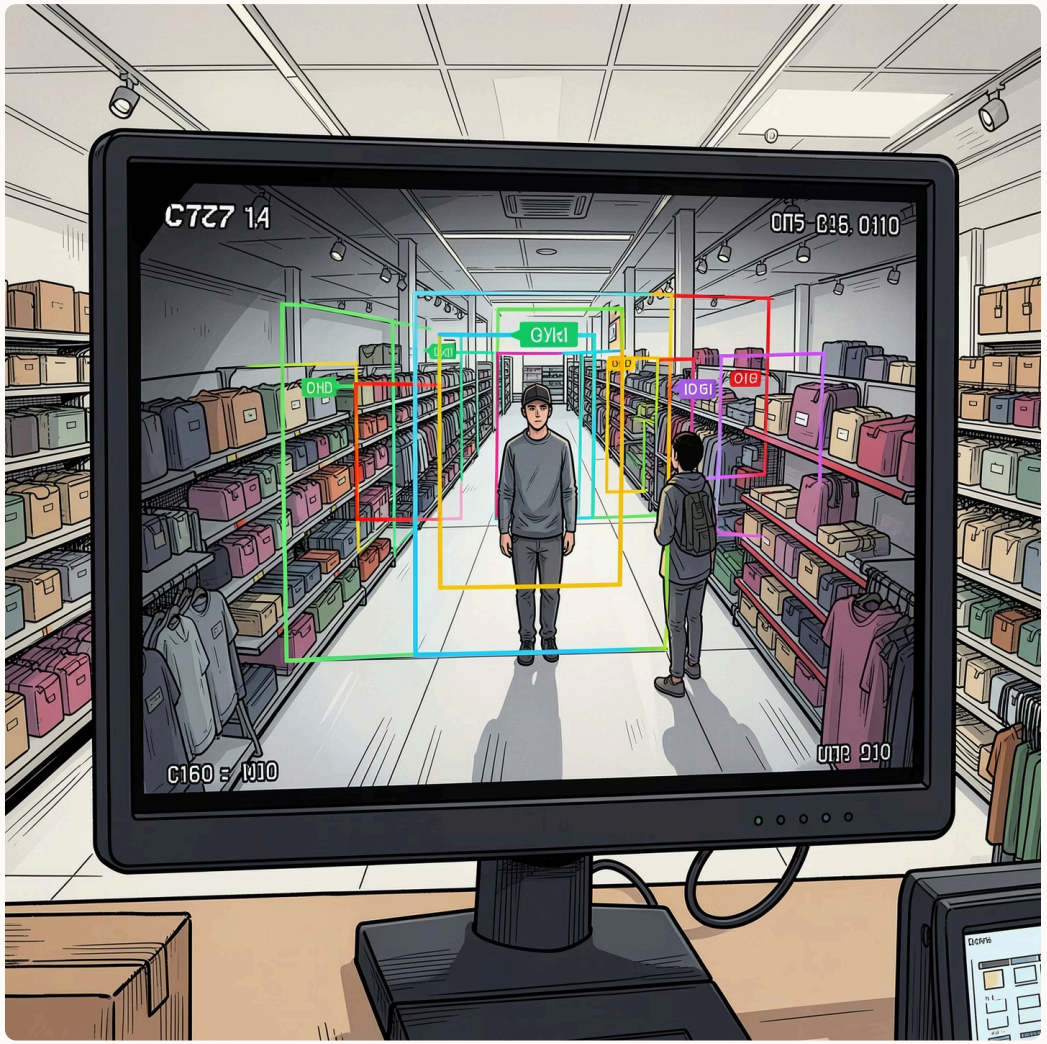
Detections are filtered exclusively for **COCO Class ID 0 (Person)** at a confidence threshold of **0.30**. This low threshold ensures occluded, side-view, and partially visible shoppers are captured while discarding background noise. Every frame is processed in under 33ms on standard edge hardware.

ByteTrack Kalman Associations

Rather than computing expensive deep-learning Re-ID embeddings, ByteTrack associates bounding boxes via **Kalman Filter motion predictions** combined with spatial IoU overlap. This keeps frame processing speeds high and track IDs stable across occlusions.

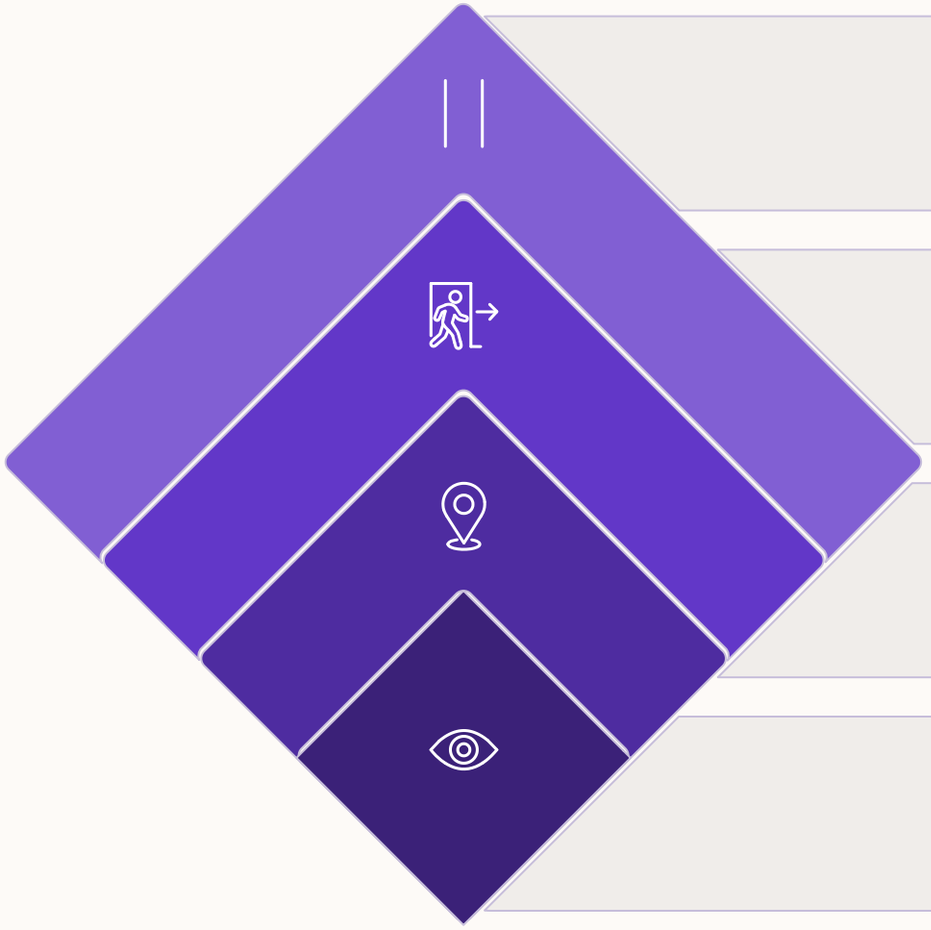
Spatial Bottom-Middle Footprint

A shopper's floor position is mapped using the bottom-middle coordinate of their bounding box: **(t_x, t_y) = (x_min + width/2, y_max)**. This anchors tracking to the shopper's feet contact point rather than a geometric centroid that shifts with head movement.



Spatial Collision & State Machine Logic

Zone membership is determined using geometric polygon containment via the Shapely library. A shopper footprint is registered inside a zone when **Point(t_x, t_y).within(Polygon(zone_coords))** returns true — enabling precise, configurable shelf and aisle boundaries defined by store managers.



Track Loss

Emit STORE_EXIT after 30 frames

Zone Exit

Emit ZONE_EXIT and dwell time

Zone Entry

Emit ZONE_ENTER on polygon hit

First Seen

Emit STORE_ENTRY event

This deterministic state machine ensures every customer journey is captured end-to-end — from store entry through zone transitions to exit — with precise dwell-time calculations at each zone boundary crossing.

REST API Documentation

StoreIntel exposes a clean, typed REST API for telemetry ingestion and analytics retrieval. All endpoints are served asynchronously via FastAPI, ensuring the API thread remains responsive even during high-volume CV processing.

POST /events/ingest

Receives telemetry payloads from the computer vision thread. Stores structured event records into SQLite with full metadata.

GET /metrics

Aggregates total footfall, current store occupancy, and popular shelf zones. Powers the dashboard's real-time KPI cards.

GET /funnel

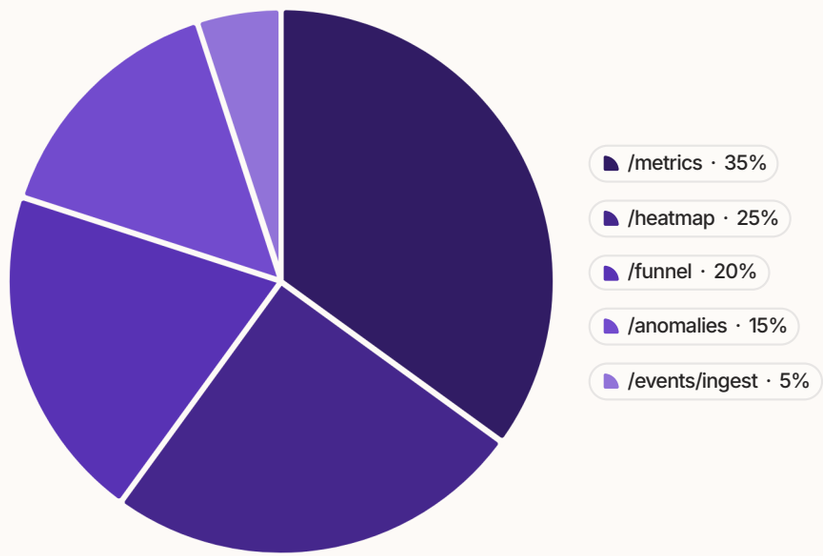
Builds conversion metrics across the customer journey: **Entrance → Shelf → Checkout**. Visualizes drop-off at each stage.

GET /heatmap

Returns coordinate density mapping for spatial heatmap rendering. Enables visual identification of high-traffic zones.

GET /anomalies

Serves active security alerts — queue congestion, loitering flags, and CCTV outage warnings — for the diagnostics panel.



SQLite Database Schema

Events Table

All spatial events are persisted in a single normalized `events` table. The schema is intentionally flat for fast aggregation queries without complex joins.

```
CREATE TABLE events (
  id          TEXT PRIMARY KEY,
  event_type  TEXT,
  timestamp   TEXT,
  customer_id TEXT,
  zone_name   TEXT,
  x           REAL,
  y           REAL,
  metadata    TEXT
);
```

Schema Design Rationale

- **id** — UUID ensures deduplication across retries
- **event_type** — STORE_ENTRY, ZONE_ENTER, ZONE_EXIT, STORE_EXIT
- **customer_id** — Persistent ByteTrack ID across the session
- **x, y** — Floor coordinate for spatial replay and heatmaps
- **metadata** — JSON blob for extensible future fields

SQLite was chosen for zero-configuration deployment — no external database orchestration, no connection pooling, no migration tooling required.

Operational Anomaly Detection

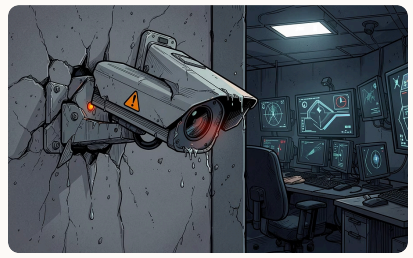
StoreIntel monitors three critical operational states automatically, writing high-severity alerts to the database and surfacing them on the dashboard diagnostics panel without manual intervention.

Queue Congestion Alert

Evaluates unique active Customer IDs inside the checkout polygon over the last **2 minutes**. If count exceeds **3 customers**, a high-severity anomaly is written — enabling proactive staff dispatch before complaints arise.

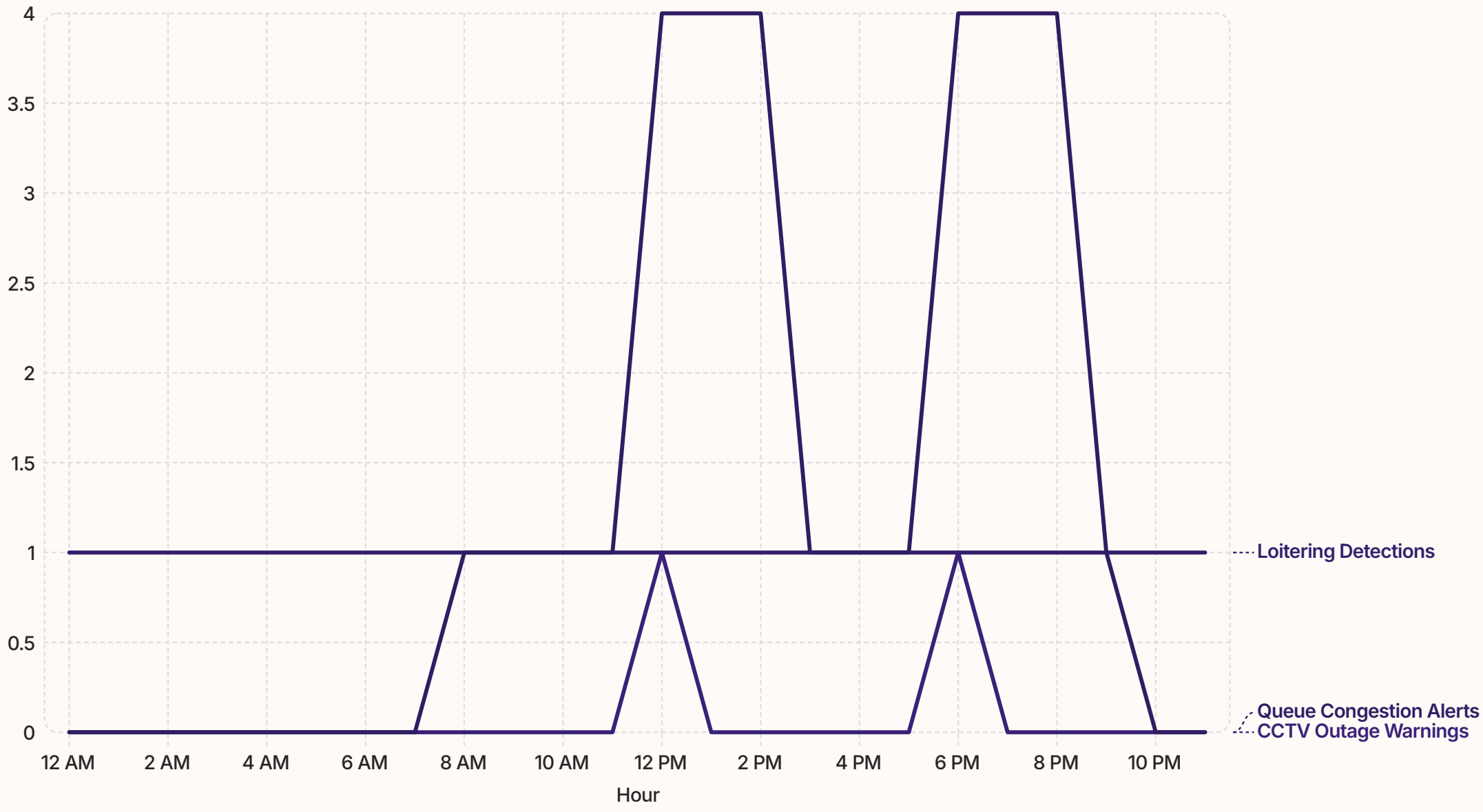
Customer Loitering Detection

Triggers when cumulative dwell time inside a shelf zone exceeds **15 seconds** (representing a real-world threshold of ~10 minutes). Flags potential loitering or customers requiring staff assistance.



CCTV Outage & Failure

Raises a system warning if elapsed time since the last received event exceeds **180 seconds**. Ensures the operations team is immediately notified of camera or pipeline failures.



The line chart highlights daily monitoring patterns: queue congestion spikes during lunch and evening rushes, loitering remains steady across the day, and CCTV outages stay rare with only two incidents.

Architecture Design Trade-Offs

Decoupled Thread Architecture

Computer vision execution runs inside an **isolated background worker thread**, completely separate from the FastAPI serving thread. This ensures API latency remains sub-10ms even during peak frame processing — the analytics dashboard never blocks on CV inference.

SQLite Over PostgreSQL

For a single-store edge deployment, SQLite eliminates database orchestration overhead entirely. There are no connection pools, no migration scripts, and no external service dependencies. The entire system is **self-contained within the Docker Compose stack**.

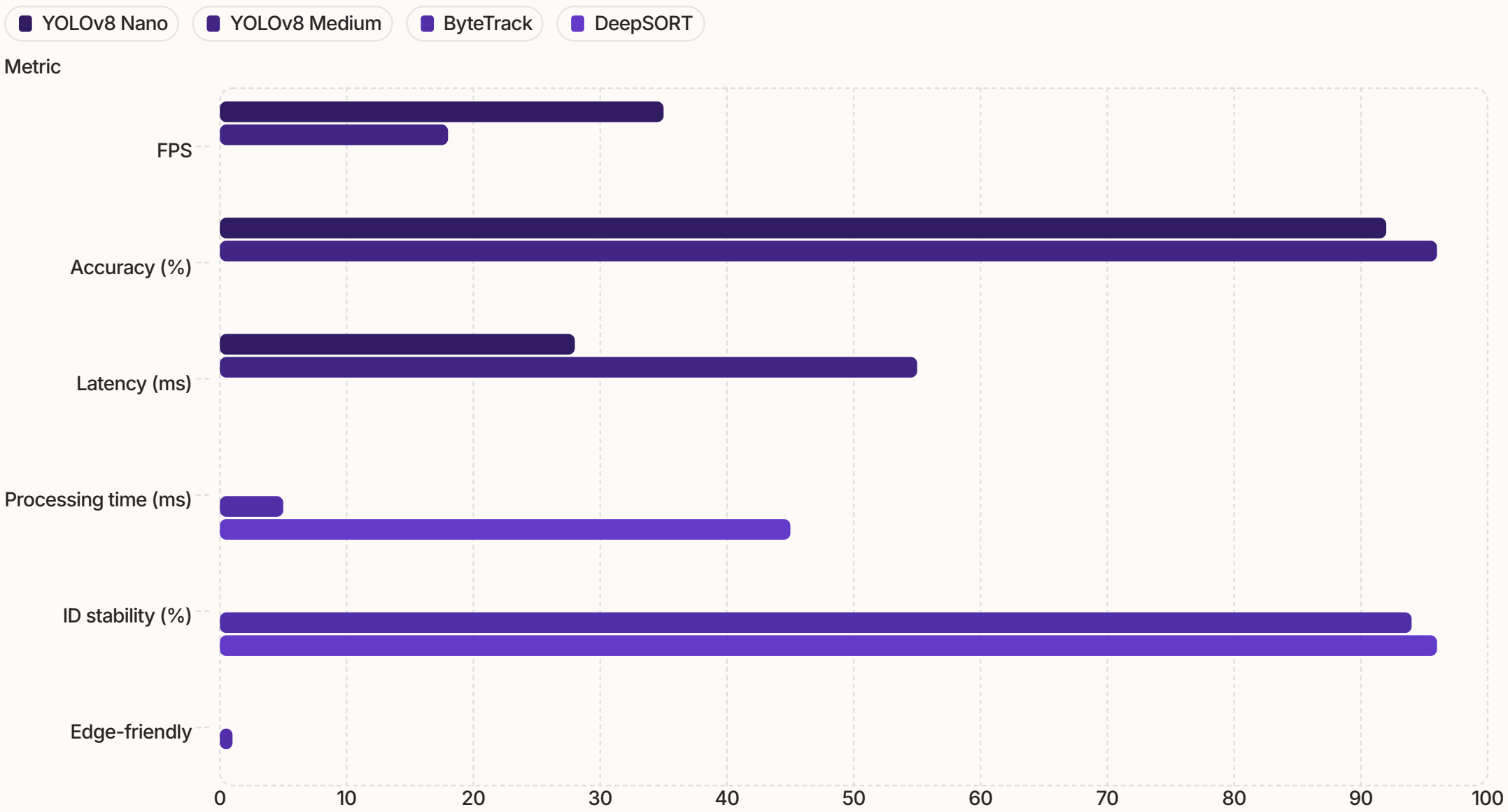
Why YOLOv8 Nano?

YOLOv8 Nano is the smallest profile in the YOLOv8 family, optimized for edge CPUs without GPU acceleration. At **30+ FPS** on modest hardware, it delivers real-time person detection with sufficient accuracy for retail spatial analytics — a deliberate trade-off favoring throughput over marginal mAP gains.

Why ByteTrack Over DeepSORT?

DeepSORT requires a Re-ID embedding network, adding ~40ms per frame. ByteTrack's **Kalman + IoU association** achieves comparable ID stability at a fraction of the compute cost — critical for edge deployment on CPU-only hardware.

Performance Comparison



The grouped bar chart highlights the main architecture trade-offs: YOLOv8 Nano prioritizes speed and lower latency, while YOLOv8 Medium improves accuracy at the cost of throughput. Likewise, ByteTrack is far more edge-friendly and faster to run than DeepSORT, even if DeepSORT offers slightly higher ID stability.

Thank You

Transforming Retail with Spatial Intelligence

StoreIntel demonstrates that existing CCTV infrastructure — when paired with modern edge AI — can deliver e-commerce-grade analytics to physical retail. No new hardware. No cloud dependency. Just intelligence at the edge.

🔗 GitHub Repository

github.com/shivamgiri068/StoreIntel

📧 Contact Email

shivamgiri068@gmail.com

📦 Submission Package

Verified production ZIP: **StoreIntel_Submission.zip**

PURPLLE TECH CHALLENGE 2026

ROUND 2 — TECHNICAL PROPOSAL

SHIVAM GIRI · FULL-STACK & AI ENGINEER